

```

) INHERITS (orders);

CREATE TABLE orders_part_2010 (
    CHECK (order_date < DATE '2011-01-01')
) INHERITS (orders);

```

Here, the partition *orders_part_2011* will hold all data from 2011 onwards, whereas *orders_part_2010* will hold all previous data. Note that I am not using any primary key clause, as *order_date* cannot be a primary key in this particular set-up. Further, always make sure that you match the data-type of values used with that of the column referred to in the *CHECK* constraints.

Optionally, you may create indexes on the partitioning key columns for each of the partitions:

```

CREATE INDEX orders_part_2011_idx ON orders_part_2011(order_date);
CREATE INDEX orders_part_2010_idx ON orders_part_2010(order_date);

```

The partitioning scheme is not yet ready. What I really want is that the data inserted into the master table should be redirected to the appropriate partition. This is where I need a trigger or rule. Triggers and rules have their own pros and cons. Here I use triggers, with trigger functions. Again, a trigger function can be written in many ways, depending on the requirements. I will show two ways of writing one, along with their advantages and disadvantages. Let's create a trigger function as shown below:

```

CREATE OR REPLACE FUNCTION orders_insert_simple() RETURNS TRIGGER
AS $$
DECLARE
    vsql Text;
BEGIN
    vsql := 'INSERT INTO orders_part_' || to_char(NEW.order_date,
'YYYY') ||
' VALUES (' || NEW.id || ', ' || quote_literal(NEW.
address) || ', ' || quote_literal(NEW.order_date) || ')';
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

Now we need to attach this trigger function to a trigger, to handle the INSERT on the master table:

```

CREATE TRIGGER orders_insert_trigger
BEFORE INSERT ON orders
FOR EACH ROW EXECUTE PROCEDURE orders_insert_simple();

```

There are quite a few interesting things used in the trigger function that you need to closely look at. It is important to note that the partition name is computed on the fly. This works for our particular case, as all the required partitions were created beforehand. This function

would not require any updates, even if you add new partitions for the next year, and so on.

Looks easy? Then take a close look again, at the part where I have used *quote_literal* to quote the input values. Essentially, you need to quote each and every possible value that is going to be inserted into the partitions. Similarly, you would need to take care of *NULL* values. This is the most cumbersome part of using this easy-to-write trigger function, so let's try out another way of writing the trigger function:

```

CREATE OR REPLACE FUNCTION orders_insert() RETURNS TRIGGER AS
$$
BEGIN
    IF (NEW.order_date >= DATE '2011-01-01' AND NEW.order_date <
DATE '2012-01-01') THEN
        INSERT INTO orders_part_2011 VALUES (NEW.*);
    ELSEIF (NEW.order_date < DATE '2011-01-01') THEN
        INSERT INTO orders_part_2010 VALUES (NEW.*);
    ELSE
        RAISE EXCEPTION 'Date out of range. check orders_insert()
function!';
    END IF;
    RETURN NULL;
END;
$$
LANGUAGE plpgsql;

```

```

CREATE TRIGGER orders_insert_trigger
BEFORE INSERT ON orders
FOR EACH ROW EXECUTE PROCEDURE orders_insert();

```

As you can see, we are now explicitly checking *order_date* to redirect data into the appropriate partitions. This doesn't need handling of quote or *NULL* values—at least, for non-partition-key values. I have simply used *NEW.** instead of referring to individual column values.

In case you are not able to create all partitions beforehand, you would need to update the above trigger function with the logic to create partitions on-the-fly. Generally, we use separate triggers for partition creation and data redirection. Obviously, the partition creation trigger should always be fired before the redirection trigger. You should take care to avoid race conditions between those two triggers.

Now that the trigger is in place, let's insert some records and verify the set-up:

```

pg=# INSERT INTO orders VALUES(1, 'pune', '2011-08-22');
INSERT 0 0
pg=# INSERT INTO orders VALUES(2, 'pune', '2010-02-22');
INSERT 0 0
pg=# UPDATE orders SET address = 'bengaluru' WHERE id = 2;
UPDATE 1

```